

Project 1: Image Manipulation

1. Overview

My goal in this assignment was to manipulate images using a variety of different functions. However, instead of worrying about manipulating these images as a `BufferedImage`, I was able to manipulate them as a matrix of integers representing pixels instead. This simplified the problem down because of my prior familiarity with matrices, or 2D arrays.

I will broadly explain some goals I had for this assignment. Many of the methods involved dealing with color (red, green, blue). Another thing I did with color was translating reds, greens, and blues into blacks and white to create a monochrome image. Beyond color, the assignment included flipping the image both horizontally and vertically. Doing this required some thought, especially due to edge cases (more on that later). Lastly, I also manipulated image size by growing and shrinking images, which also proved tricky because of edge cases.

2. Description

Each effect is a subclass of `ImageEffect` and implements a similar `apply` method. Each `apply` method receives a 2D int array that represents pixels, along with an `ArrayList` of parameters. However, not every transformation requires using the parameters: in fact, only the threshold transformation does. Each `apply` method returns another 2D array representing pixels. Usually, this array is the same size as the input array, but in the case of the `Grow` and `Shrink` transformations, a different size matrix is returned.

The six transformations dealing with color removal work relatively similarly: `NoRed`, `NoBlue`, `NoGreen`, `RedOnly`, `GreenOnly`, and `BlueOnly`. The algorithm traverses through the entire matrix, and for each pixel, we create a new pixel that satisfies the transformation. If we are applying the `NoRed` transformation, each pixel would be traversed, and we would make a new pixel in its place with the same green and blue values, but a value of 0 for red. Similarly, for the `RedOnly` transformation, we traverse through each pixel and set it equal to a new one with the same red value, but a 0 green value and a 0 blue value.

The `BlackAndWhite` transformation iterates through each pixel and computes the average value of the red, green, and blue values. After computing this average, a new pixel is made with a red, green, and blue value all set equal to the average value. Because this is a division among ints, any fraction is simply discarded.

The VerticalReflect transformation flips the image across the middle vertical line. It processes each row separately and compares corresponding pixels from both sides of the row. For example, the first is swapped with the last, the second is swapped with the second last, and so on. It only iterates through half of each row because the second half is completed through swaps. Also, if a row has an odd number of pixels, the middle pixel simply stays the same.

The Horizontal transformation reflects the image across the middle horizontal line. It compares corresponding rows from the top and bottom of the image. For example, the first is swapped with the last, the second is swapped with the second last, and so on. It only iterates through half of the rows because the second half is completed through swaps. Also, if the image has an odd number of rows, the middle row simply stays the same.

The Grow transformation returns an image that is twice as long and twice as wide. It processes each original row and creates two additional rows for it. Then, for each original pixel, it creates 4 duplicates of it in a 2x2 block in the two new rows. This approach works for the edge case where an image is jagged because it creates the two new rows with twice the length of the original row that it's currently processing.

The Shrink transformation creates an image that is half as long and wide. It works by processing 2x2 pixel blocks. For each block it processes, the algorithm averages the red values together, the green values, and the blue values. Then, in the smaller image, it creates a pixel that uses each averaged value: red, blue, and green. By doing this, the 4 pixels in the 2x2 block are averaged into just one pixel in the new image matrix. For the edge case in which the image has an odd number of rows/columns, the extra row/column is ignored. As for jagged rows, it uses the smaller width of each pair of rows so it only processes pixels that exist in both rows.

Finally, the Threshold transformation simplifies the image down to just eight possible colors. This is done by checking the red, green, and blue values of a pixel separately. If the value is less than the threshold, it is set to 0. On the other hand, if it is equal to or greater than the threshold, its value is set to the full 255. This process is repeated for each pixel in the input array, and the pixel is replaced with a new pixel using only RGB values of 0 and 255, according to the threshold comparison. This is the only transformation that uses the parameter. This is done by creating the parameter ArrayList and then adding an ImageEffectParam object to the list. Then, this object's 'value' can be modified to change the threshold the code considers.

As for assumptions, the code does not assume that an image is a square or that the dimensions are a power of 2. Most transformations process the image using its actual number of rows and columns. For instance, odd dimensions are handled by leaving a middle pixel or row unchanged after reflecting it, ignoring leftover pixels during shrinking, and doubling dimensions regardless during growth. Jagged rows are also supported by the transformations, which is

mostly relevant for reflection, growth, and shrinking. Shrink uses 2x2 blocks and uses the shorted width of the row pair. The threshold transformation handles values below or above 255 through a simple validation check (in both cases, we use the default value of 127). Even though one cannot enter one of these values through the GUI, it is still possible in testing, so I added this check. Some assumptions that are made, however, are that the array is not null and contains valid rows with usable pixel values (get color functions work, etc.). The code does not handle cases where these assumptions do not hold. Also, I have left invert untouched, so it may not account for the edge cases other transformations handle.

3. Discussion

The scope of the solution includes the 12 required image transformations. These transformations can be used to create interesting images, as shown in the next section. The transformations can even stack, allowing for more complicated filters and changes to the image. The transformations work on 2D arrays of ints, representing pixels.

The quality of the solution is high, as each transformation has been rigorously tested through unit tests and black-box testing. Each transformation works as intended as all 21 test cases pass; moreover, the transformations appear to work visually.

The most interesting result is dealing with jagged images. Jagged images rarely exist in the real world, so working with them in this project was quite interesting. For example, when growing a jagged image, the result just “seems” unusual to the human eye, mainly because we are not used to observing images like them in the real world. Nonetheless, working with them here has been a novel and beneficial experience.

Beyond considering edge cases, another problem I encountered was handling threshold values not in the allowed $[0, 255]$ range. At first, I did not think this was possible because the GUI does not allow for it. But after more consideration, I realized that it was possible through internal testing, and I wrote a simple check to ensure that the value defaults to 127 if it is outside of the allowed range.

I was not able to complete any enhancements or extra credit for this project, mainly because I was busy with club applications. But, after a quick read of the Karma assignment, it is definitely something I plan to implement in future projects.

I have completed a lot of programming assignments in high school, but none of them required me to think about edge cases to the level this project did. For example, most previous projects included a lot of really helpful assumptions, which allowed me to overlook complications in edge cases. Going into this project, I never considered edge cases like jagged

arrays. But after diving deeper into the project, I learned that this was a possibility and that my code needed to handle it. So, the existence of more complex edge cases was one of the biggest surprises and learning experiences for this project.

4. Testing

I tested each transformation using JUnit unit tests with small (but intentional) images. Each test compares the red, green, and blue values of every output pixel with the expected result. I also tested important edge cases, like odd/even dimensions, jagged rows, leftover pixels in shrinking, custom threshold values, and invalid threshold values.

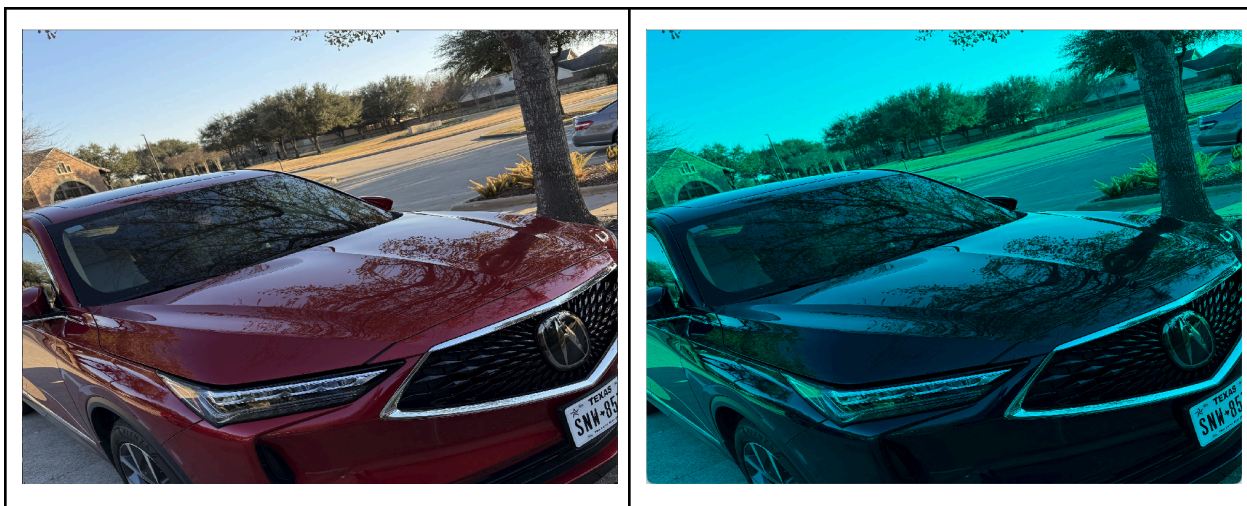
Of course, some images are more useful for specific effects than others. For example, an image containing a lot of red is useful for testing NoRed, while a completely black image is useless. Similarly, images with diverse colors are useful for testing the BlackAndWhite transformation. Finally, images with high symmetry are not helpful for testing reflection transformations.

The tests I wrote considered which effects an image can meaningfully test. A colorful image is best for removal, black/white conversion, and thresholds, while an image with different patterns is good for reflection. My testing gives me strong confidence in my code because every required transformation has a robust test (some with multiple), and each one passes. However, testing someone else's code might not prove it works for all inputs. Confidence could be increased by stacking transformations (applying multiple) or using larger, real-world images.

Here are some fun real-world images I transformed using my code. The left image is before, and the right is the after. Underneath describes the transformation(s) used.



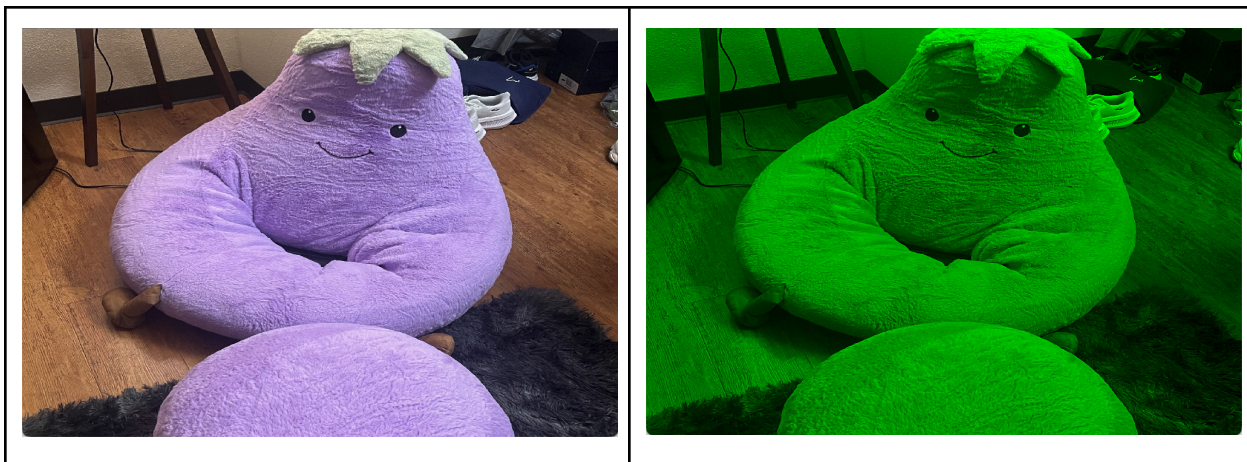
HorizontalReflect (not very useful!)



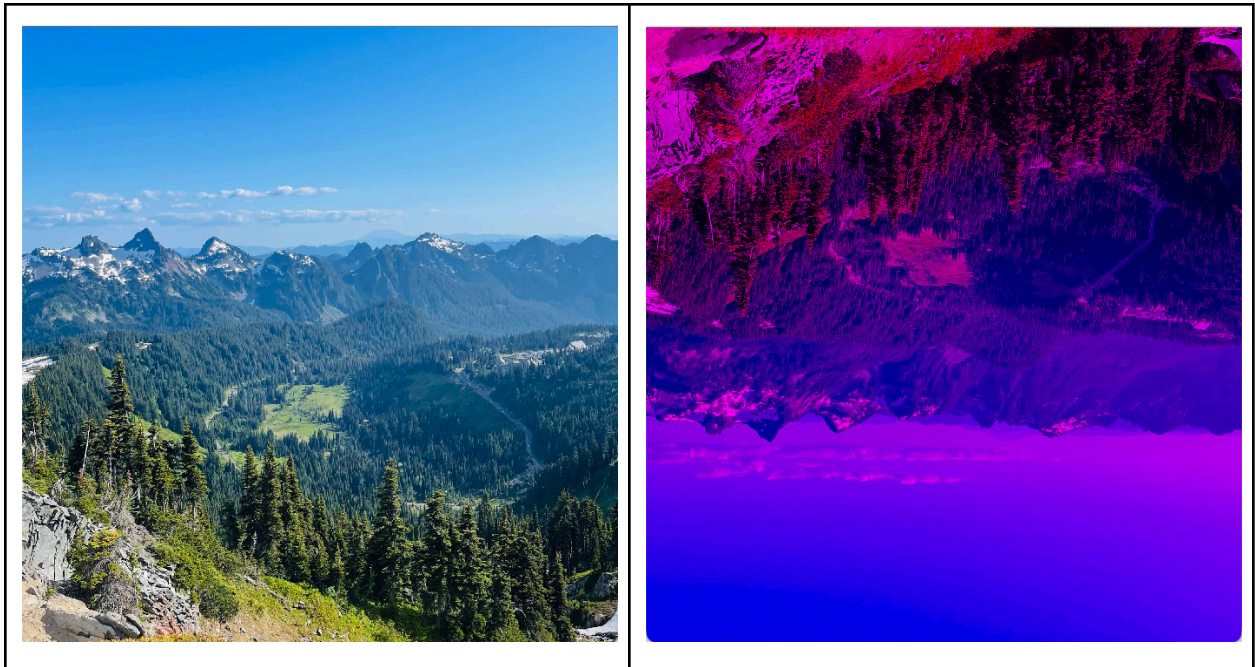
NoRed



BlackAndWhite



GreenOnly



NoGreen, HorizontalReflect